

100 points possible.

Name: Josiah Schmitz

***Submit your pdf answer file and two code files, your Ruby and Python codes.**

Part1-Ruby: 50 pts

1. (12 pts) Write a Ruby regular expression that would match each of the following types of strings.

a) (4 pts) Strings that contain exactly two integer numbers (each number may have any number of digits). The string may contain other words/characters as well.

(Examples: "There were 20 dogs in 5 cars." Or "There is 1 barn on the 237 acres")

*Add your code to the Ruby file.

b) (4 pts) Strings that contain no numeric (digit) characters.

*Add your code to the Ruby file.

c) (4 pts) Strings that start with the letter "s", followed by two vowels, and that have the same two consecutive vowels later in the string: (for example: "sourdough").

*Add your code to the Ruby file.

2. (7 pts) Suppose I have an object of class Automobile called car1, and I call the method car1.horsepower(). Describe the process that Ruby uses to search for the horsepower() method. What happens if there is no such method? How does ruby ensure that mixins will be searched in this process?

Answer: Ruby recognizes 'car1' as the receiver of class 'Automobile'. It then searches all the methods belonging to 'Automobile' that are created at runtime until it finds horsepower() and calls whatever is inside horsepower(). If the called method can't be found, Ruby will raise a 'NoMethodError' to alert the user that the method doesn't exist. Ruby can find mixins by creating a lookup chain to try to find all methods with that name that are native to the class or inherited by it.

3. (8 pts) Could you write a Ruby-style *iterator* in Racket? Why or why not? (*Hint: first define what exactly an iterator is in Ruby. Once you have done this, ask yourself “Could I create one of those in Racket?” If the answer is yes, give an example or show how. If the answer is no, what is Racket missing that you would need to do it?*)

Answer: Yes, you can implement a Ruby-style iterator in Racket. This is primarily because Racket has the capability to pass a block of code acting as a function as a method argument. This can be used like Ruby because a Ruby iterator is essentially passing a block of code to an existing method and applying that code to each element of a collection inside the method (using the ‘yield’ command, specifically). An example the textbook uses in Ruby is as follows:

```
a = [1, 2, 3, 4, 5]
a.each do |k|
  print k, " "
end
```

A corresponding example in Racket would be:

```
(define a-list '(1 2 3 4 5))

(for-each
 (lambda (given)
  (printf "~a " given))
 a-list)
```

Both examples loop through elements in a list and iteratively apply a print function to each element.

4. (7 pts) What are *dynamic methods* in Ruby? Give two examples of situations in which dynamic methods are useful.

Answer: Dynamic methods are methods whose name and function are determined dynamically. ‘Dynamically’ here means that instead of being hard-coded, these attributes can be determined by other aspects of the class/methods. One situation where this can be useful is if you need a bunch of very similar methods that do nearly the exact same task. By making them dynamic, you can make the methods’ operations determined by what the name of the method is, saving you from having to write nearly identical code for each individual method. Another example might be wanting to just slightly alter an existing method inherited in a class. Using a dynamic method would allow you to simply modify the output of the existing method, instead of completely overwriting it yourself.

-
5. (9 pts) Write a Ruby mixin called *Countable* that adds the method *count* to any class. The *count* method should return the number of items enumerated by the classes *each* method. If included in a class that does not provide *each*, calling *count* should return zero.

*Add the code to the Ruby file

6. (7 pts) There are two main ways in which computer scientists describe the variable-typing policies of languages: strong vs. weak, and dynamic vs. static

Give an example of a strongly-typed language and a weakly-typed language and describe what it is about the language that makes it strongly or weakly typed.

Answer: Java is considered a strongly-typed language, and JavaScript is considered a weakly-typed language. Java is strongly-typed because a variable cannot change its type once it has been set, except via type casting (which is really just converting a variable's type and assigning the result to a new variable). JavaScript, on the other hand, is weakly-typed because a variable can change to a different type even after it was initially assigned, making that single variable's type much more flexible.

Part2-Python: 50 pts

7. (3 pts) In Tri-color algorithm every object in the system is “marked” in one of three colors, White, Gray, or Black. In your own words, describe the Tri-color algorithm, its initialization and the tracing steps, and then color each object in the system. I-Tri-color algorithm:

The general idea is....

The Tri-color algorithm is a way to mark objects for garbage collection using three categories represented by the colors white, gray, and black. By marking objects with these colors, the system can determine what objects should/shouldn't be recycled. This keeps unneeded objects from wasting memory in the system.

White: objects that may be recycled

Gray: objects that won't be recycled, but potentially refer to white objects

Black: objects that won't be recycled and don't refer to white objects

II-Initialization:

Before the algorithm starts...

All objects are placed into either the white or gray sets, with the black set remaining empty. The grey set will hold any objects that are used by the main program (roots), and the white set will hold all other objects.

White: all objects that aren't roots

Gray: the roots

Black: nothing

III-Describe the Tracing steps:

The algorithm will cycle through all objects in the grey set and apply the following 2 steps: Any white objects that the selected grey object references will be marked as gray. The selected grey object will then be marked as black. These 2 steps will be repeated until all objects are either white or grey. All white objects can then be safely recycled.

After the Tracing is done:

White: objects that will be recycled

Gray: empty

Black: objects that are safe from recycling

8. (6 pts) Suppose you are given the following segment of C++ code:

```
int global1;
int *global2;

int sub1 (int x) {
    int sub1var1;
    int *sub1ptr1;

    sub1ptr1 = new int;
    ... more code ...
    return sub1var1;
}

int main () {
    int mainvar1, mainvar2;
    float mainvar3;
    int *mainptr1;
    double *mainptr2;

    mainptr1 = new int;
    mainptr2 = new double;
    mainvar1 = sub1(mainvar2);
    ... more code ...
    return;
}
```

a- (2 pts) If the program is paused for garbage collection, using a conservative mark-and-sweep garbage collection algorithm, while it is in `main()` just before calling `sub1()`, what set (black, white, or gray) will each object in the program be in when garbage collection starts? For pointers, include both the pointer itself and the object/data it is pointing to.

White: empty

Gray: `global1`, `*global2`, `mainvar1`, `mainvar2`, `mainvar3`, `*mainptr1`, `*mainptr2`

Black: empty

b- (2 pts) Suppose that the garbage collector begins the tri-color algorithm by processing every item in the root set first. After all of the roots are processed, what set will each of the variables belong to?

White: empty

Gray: empty

Black: `global1`, `*global2`, `mainvar1`, `mainvar2`, `mainvar3`, `*mainptr1`, `*mainptr2`,

c- (2 pts) What if the program is paused for garbage collection while in `sub1()`, just before executing the return statement? At the start of GC, what set will each variable belong to?

White: `sub1var1`, `*sub1ptr1`

Gray: `global1`, `*global2`, `mainvar1`, `mainvar2`, `mainvar3`, `*mainptr1`, `*mainptr2`

Black: empty

9. (3 pts) Write a short (a line or two) Python program that demonstrates that Python is introspective. Write your short answer here and add any example code to the Python file:

Answer: Python is introspective because it can see what type an object is at runtime. This can be done by using the **`type()`** function which will return the type of an object when called (see Python file).

10. (3 pts) Write a short (a line or two) Python program that demonstrates that integer variables are actually objects. Write your short answer here and add any example code to the Python file:

Answer: An integer variable can be checked to see if it's an object by determining if said variable is an instance of a class (in this case, class `int`). All instances of a class are objects, by definition, so using the **`isinstance()`** method will show us if an integer variable is an instance and, consequently, an object (see Python file).

11. (3 pts) Write a short (a few lines) Python program that demonstrates that functions are first-class objects in Python. Write your short answer here and add any example code to the Python file:

Answer: For a function to be a first-class object in Python, it has to be capable of all the basic tasks of other objects in Python, like being able to be assigned as a variable or being returned from a function. Examples of this are shown in my Python file.

12. (3 pts) Write a short answer (a few lines) to explain what is a destructive method. What is the difference between `sort()` and `sorted()` (show examples). Write your short answer here and add any example code to the Python file:

Answer: A destructive method is a method that alters the attributes of an object when called. For example, calling **`append()`** to a list will add an element to the list. However, this is done by altering the original list itself instead of returning a new list and leaving the original intact. This makes **`append()`** a destructive method. By this same logic, **`sort()`** is destructive because it sorts the given list and returns `None`. **`Sorted()`**, however, is non-destructive because it returns a new list that is simply *equivalent* to the given list when sorted (see Python file).

13. (3 pts) Write a short answer (a few lines) to explain what is list comprehension in Python and give an example with multiple conditions. Write your short answer here and add any example code to the Python file:

Answer: List comprehension is a way to create lists in Python by writing a small function in the list creation itself. This function can be used to add a series of elements that are the result of some kind of operation to the list. List comprehension makes code more concise and efficient by eliminating excess loops or variables that would otherwise have to exist outside of the list's initialization (see Python file).

14. (3 pts) Write a short answer (a few lines) to explain what is an iterator and what is an iterable object. Name two examples of iterables in Python. Write your short answer here and add any example code to the Python file:

Answer: Iterators are a special kind of object in Python that have a series of elements in them that can be sequentially (iteratively) operated on in various ways. Iterators always have an `__iter__` method and a `__next__` method which are used to keep track of the index of an element in the object. Two common examples of iterables in Python are lists and arrays. (no code in Python file)

15. (3 pts) Write a short answer (a few lines) to explain what is a generator in Python and how it is different from an iterator. Write your short answer here and add any example code to the Python file:

Answer: A generator is a class type that, like iterators, performs a specified operation sequentially over every element in a collection. The main difference between an iterator and a generator is that generators call `yield` to return elements in their collection instead of using the `return` keyword. This allows the state of the object to be maintained during iteration. See Python file for example of generator that yields elements of a collection in order.

16. (3 pts) Write a short answer (a few lines) to explain what is a decorator in Python and how it is used (show with an example). Write your short answer here and add any example code to the Python file:

Answer: A decorator is simply a function that takes a function as an argument and returns a modified version of the original function. See Python file for example of decorator that reverses output of a basic function that returns a name.

17. Write a Python generator that produces binary numbers as strings. In other words, if your generator is called `my_generator`, the following code:

```
for val in my_generator():  
    print(val)  
    if (val == '1000'):  
        break
```

(8 pts)

Should print the sequence: 0, 1, 10, 11, 100, 101, 110, 111, 1000

*Add the code to the Python file.

18. (9 pts) Write a Python decorator, `print_instead` that changes a function that returns a single value to instead print that value and return None. If the following simple recursive function that computes the n^{th} Fibonacci number, were decorated with `print_instead`, as follows:


```
@print_instead
def fibonacci(n):
    current = 1
    previous = 1
    second_previous = 0
    if (n <= 0):
        return 0

    for _ in range(n-2):
        second_previous = previous
        previous = current
        current = previous + second_previous

    return current
```

Then executing the line:

```
result = fibonacci(7)
```

should place the value **None** into **result** and print the value **13** to the screen.

Your decorator should work on any function, including functions that take any number (including zero) of positional and/or keyword arguments.

*Add the code to the Python file.